



Reduction of the worst-case execution time of real-time tasks targeting many-core platforms

Internship at CISTER - Research Center in Real-Time & Embedded Computing Systems (Porto, Portugal) from 2013/05/18 to 2013/08/30, presented to the jury on 2013/09/11, for the degrees of:

- *"Magistère Informatique et Télécommunications" (2nd year) from ENS Cachan, Brittany extension.*
- *"Master Recherche et Innovation" (1st year) from the University of Rennes 1.*

Abstract

Current trends in the development of processors mostly consist in multiplying the number of cores on a single chip. After the multi-core processors appeared what we call the many-core processors, due to their huge amount of cores. In this report, we consider programs with real-time needs targeting that kind of architecture. For a given program, we try to execute it as quickly as possible in its worst case, by taking advantage of the numerous cores, while allowing to keep guaranties on its worst-case execution time.

We present here a technique to transform the data flow graph representing a program in order to reduce the length of its critical paths, and thus its worst-case execution time. To do so, we detect some spots in the graph where some time might be lost because all the data is available to compute the rest of the program, but we have not enough control information to be able to process. This situation occurs for example when all the branches of a conditional structure are speculatively computed, but we still don't know the result of the test to be able to choose between different assignments of a single variable that could have been made in both branches. We present an algorithm that will apply these transformations in multiple spots along the critical paths of the program in order to reduce their lengths, and show that the worst-case execution time of the program is then reduced.

Keywords

Real-time tasks, many-core processors, worst-case execution time reduction, graph transformation, control flow graph, data flow graph.

Student: Bastien Padeloup
Director: Eduardo Tovar
Supervisor: Vincent Nélis

Acknowledgements

I first want to thank Eduardo Tovar for giving me the opportunity to come to Porto to work at CISTER on real-time and embedded systems. I also want to thank my supervisor Vincent Nélis for the help he provided me all along the internship. Additionally, I want to thank my office mates Geoffrey Nelissen and Patrick Meumeu Yomsi, and more generally all the researchers and students in the laboratory for the nice everyday life at the office and beyond. Many thanks too to Cristiana Barros and Inês Almeida for guiding me through the endless maze of administration.

I also want to thank ISEP, ENS Cachan and the University of Rennes 1 for the financial support during those last three months, which allowed me to focus on my work without any worries about funding.

Then I want to thank Isabelle Puaut who made me discover CISTER and helped me obtaining an internship here. I also want to thank Lauren Lefumeur, TODO and Isabelle Puaut again for reading and pointing out some mistakes in the previous versions of this document.

Finally, I want to thank all the people in Portugal who allowed me having a great time here, and helped me stay motivated until the end of the internship by providing me support, useful information, nice beer and great music.

Contents

Introduction	1
1 Various representations for a program	3
1.1 Control flow graph	3
1.2 Data dependencies graph	4
1.3 Data flow graph	5
1.3.1 Decomposition of the program into elementary instructions	5
1.3.2 Graph base using the data dependencies	6
1.3.3 The problem of ambiguous entries	6
2 Transformation of the data flow graph	11
2.1 Analysis of a given selector	11
2.2 Transformation of an improvable selector	12
2.3 Algorithm to improve the worst-case execution time of the program	17
Conclusion	19
References	20

Introduction

In 1965, an engineer from Fairchild Semiconductors stated in "Electronics Magazine" that the complexity of semiconductors would double every year for the same price. Ten years later, the same person revised this prediction, and foresaw that the number of transistors on a single silicium chip would double every two years. That person was Gordon Moore, one of the three founders of the Intel company, and those predictions gave birth to what was called Moore's law. Although those predictions were only empirical extrapolations of the current situation, they somewhat were impressively close to reality [1].

Moore's law was in fact used as a guideline for hardware vendors, who tried to follow it as close as possible in terms of miniaturization of the transistors. However, studies now show that we are close to reaching what is called the Wall. This frontier is the theoretical limit of miniaturization for transistors, due to the limits of current technology. For years, the evolution in technologies such as x-ray lithography allowed to follow Moore's law and *push* the Wall, but the incredibly small size of current transistors (22nm in industrial processors in 2012) is close to quantum effects. Those effects would put an end to the ongoing miniaturization, and should arise around 2015 if following Moore's law [2].

Even with those limitations, it is essential to keep improving the computing power of computer architectures. A lot of research fields such as genome sequencing, weather forecasting or counter-cryptography require high performance computers, and advances in those fields greatly depend on the increase in computation power. However, as stated before, the miniaturization possibilities are close to an end. To address this problem, the current trends in research on architectures are multi-core processors.

Multi-core processors appeared into desktop computers in 2006 with the Intel Core 2 product line. Originally composed of a single die (small block of semiconducting material) containing two cores, the number of cores increased during the last years and now quad-core processors are the norm. This increase in the number of cores on a single chip is very likely to continue for a non-determined period. Companies such as Adapteva, Altera or Kalray now propose multi-core platforms with more than 256 cores. Such processors with a massive amount of cores are generally called many-core processors.

On the software side, real-time systems are systems in which the correctness of the programs (tasks) not only depends on the results of the computations, but also on the time at which the results are produced. The deadline of a task is the moment before which every possible execution of the task should have ended. Real-time systems can be separated in two main categories: *soft* real-time systems, in which missing the deadline doesn't jeopardize the system correctness (the coffee from the machine should be ready before one minute); and *hard* real-time systems, in which missing the deadline can result in catastrophic situations (activation of the airbag, control of the cooling in nuclear plants... causing the loss of human lives, ecological or financial disasters). To ensure that the deadlines are met for all the tasks, we compute their worst-case execution time (WCET), which is the longest time before the result is produced. This can be done using various technics, such as abstract interpretation [3] or probabilistic methods [4]. A real-time system will meet its requirements if the worst-case execution of every task can be finished before its deadline.

Real-time systems are generally used in embedded platforms, and thus need to run on architectures with a low energy consumption. In this context, modern multi-core processors (and also many-core processors) start to be very interesting platforms for real-time systems. For example, the Kalray MPPA-256 processor can reach a total of more than 2 Tera operations per second with a typical 5W power consumption [5]. Moreover, a study conducted by Chuck Moore for AMD [6] shows that single-thread performance tend to stabilize, so switching to multi-threaded tasks on multi-core hardware is now essential to fulfill real-time tasks with higher computing requirements before their deadlines.

To use effectively the great number of cores available in multi-core processors, it is necessary to split the tasks that will execute on them into threads that can be run simultaneously. To achieve that, the programmer manually determines the parallel sections and either creates the threads with tools such as the pthread library [7] or the functions from the MPI [8] library, or adds annotations to the code that will be analyzed by semi-automatic tools such as OpenMP [9]. Another way to effectively split a given program into threads is to rely on fully-automatic parallelization tools, such as Pluto [10] or PIPS [11]. Those tools cover a lot of techniques, such as loop nests parallelization using the polyhedral model [12] to determine the independence of the execution of instructions in different loop iterations, or semantical analyses to improve the code [13] by transforming it into another more efficient semantically equivalent code.

However, to the best of our knowledge, those tools only aim at accelerating the average execution time of the programs, and don't focus at all on their WCET. In this report, we present a method that can be implemented in a real-time compiler in order to improve the WCET of a given task on a multi-core architecture. To do so, we use the data flow graph representation of the task, which by construction puts in evidence a lot of parallelism

possibilities. With this model, every instruction that has all of its data dependencies resolved will execute. However, for conditional structures like `if (cond) then x := a else x := b`, we must wait for the result of the condition to be available to determine which assignment of `x` was the correct one before continuing the execution. This may result in some lost time waiting for the condition even if all the code in the two branches is computed. To determine if a loss of time might occur, we compute the longest time before the test and the branches are computed. If we detect that the test might be finished after the branches in the worst case, we apply some graph transformations to duplicate the instruction following the ambiguous assignment in both branches of the test. We then update the time analysis, and continue copying code until there is no possible time lost. With such transformations made offline (at compilation time because of the time constraints), we remove the situations that can make a time loss occur along the critical paths of a task, and therefore we improve its WCET.

This report is organized in two sections. We first present in section 1 three models for a program that are necessary to take profit of the possible parallelism of the program, and to keep track of its possible executions on a typical Von Neumann architecture [14]. Then, in section 2, we present the detection of the parts of the data flow graph representing a program that can be improved, and the transformations we apply to reduce the execution time of some code. We then present an algorithm to apply those transformations along the critical paths of the program in order to reduce its WCET.

1 Various representations for a program

In this section, we present the most commonly used graph representations to model behaviors of a program. We first introduce in section 1.1 the control flow graph, describing the internal organization of a program. Then we present in section 1.2 the data dependencies graph, which puts in evidence the precedence constraints among the computations and gives us a base for the construction of the data flow graph in section 1.3. This latter representation will be our main model in the next sections.

A program P is composed of a set of instructions I and a set of variables X . In the next graph constructions, we will use this definition as a description of the program to analyze.

1.1 Control flow graph

The control flow graph of a program is a modelization of the possible execution paths in a program. In this graph, a node represents an instruction in the code¹, and an edge from a node n to a node n' represents the fact that there exists an execution path on a fully-sequential machine along which the instruction represented by n' is executed directly after the one represented by n .

In a more formal way, the control flow graph $\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$ of a program $P = (I, X)$ is built as follows (elements relating to the control flow graph will have the letter c as an index):

- $\mathcal{V}_c$ is the set of nodes of the graph $\mathcal{G}_c$.
- $\forall i \in I : \exists! v_c^i \in \mathcal{V}_c$.
- $\mathcal{E}_c$ is the set of edges of the graph $\mathcal{G}_c$. An edge $e_c^{ij} \in \mathcal{E}_c$ from $v_c^i \in \mathcal{V}_c$ to $v_c^j \in \mathcal{V}_c$ is a tuple (v_c^i, v_c^j) .
- Let $\text{next_pc}(i \in I)$ be the set of instructions of I that can be executed immediately after i (*i.e.* the instructions located at the possible values of the program counter once i is executed, for all possible states of the variables from X when executing i). $\forall i \in I, \forall j \in \text{next_pc}(i) : \exists! e_c^{ij} = (v_c^i, v_c^j) \in \mathcal{E}_c$.

In this control flow graph, we consider two kinds of nodes: test nodes (representing code returning a boolean, such as $b < a$ in figure 1) and instruction nodes (the others). Those nodes are all elements of $\mathcal{V}_c$, but making this distinction will be useful for the data flow graph definition in section 1.3.3.

Moreover, we call entry node (or input node) the node that is the first to execute. We represent it as the target of an edge with no source (For example, $a := 0$ in figure 1). The same way, we define an exit node (or output node) as a node that can be the last to execute. It is represented as the source of an edge with no target (For example, $c := a$ in figure 1). A sequence of nodes in the control flow graph, from the entry node to an exit node, following the feasible edges², is called a path, and represents a possible execution of the program represented by the graph. Iterating over all possible paths in the graph is strictly equivalent of iterating over all possible execution paths of the program it represents [15].

Figure 1 presents a simple program. In this example, there is only one possible path from the entry node to the exit node: $a := 0$; $b := 5$; $b < a$; $a := b$; $c := a$. Therefore, on a sequential machine, such a program would take 5 clock cycles to execute (if we assume that every instruction can be performed in one cycle).

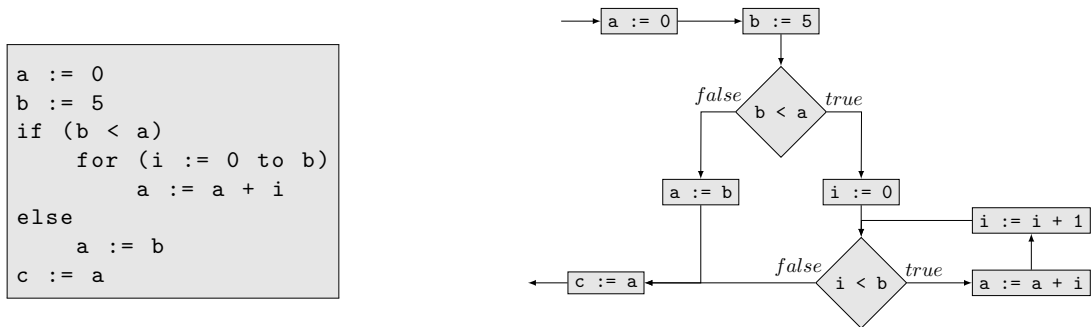


Figure 1: Example code (left) and its corresponding control flow graph (right).

¹Most representations of control flow graphs use the nodes to represent basic blocks. This is not the case here because we need a finer grain representation for the transformations presented in section 2.2.

²An edge is feasible if, in the context of the subpath that is already executed, it can be taken. For example, if $a = 1$, $b = 2$ and we have to execute instruction $a < b$, then only the edge leading to the first instruction of the **then** block is feasible.

1.2 Data dependencies graph

Contrary to the control flow graph, the purpose of the data dependencies graph is not to represent the behavior of a program on a sequential machine, but the computations that must be done first in order for the variables to be set before being used by other instructions.

In this graph too, a node represents an instruction in the code. However, edges now represent data dependencies. We say that node n is data-dependent on node n' about variable x if n' assigns a value to x , and there exists a path in the control flow graph along which instruction n uses the value of x set by n' . So if there exists a data dependency for node n about variable x set by node n' , then there is an edge from node n' to node n in the graph, labelled with variable x .

In a more formal way, the data dependencies graph $\mathcal{G}_d = (\mathcal{V}_d, \mathcal{E}_d)$ of a program $P = (I, X)$ is built as follows (elements relating to the data dependencies graph will have the letter d as an index):

- $\mathcal{V}_d$ is the set of nodes of the graph $\mathcal{G}_d$.
- $\forall i \in I : \exists! v_d^i \in \mathcal{V}_d$.
- $\mathcal{E}_d$ is the set of edges of the graph $\mathcal{G}_d$. An edge $e_d^{ijx} \in \mathcal{E}_d$ from $v_d^i \in \mathcal{V}_d$ to $v_d^j \in \mathcal{V}_d$ labelled with variable $x \in X$ is a tuple (v_d^i, v_d^j, x) .
- Let $i \in I$ such that i assigns a value to variable $x \in X$. Let $\text{next_data}(i, x)$ the set of instructions of I that need variable x and can use its assignment from i before it is assigned again (for all instruction $j \in I$ such that $j = f(x)$, $j \in \text{next_data}(i, x)$ if there exists an execution path in P along which j uses the value of x set by i). $\forall i \in I, \forall x \in X, \forall j \in \text{next_data}(i, x) : \exists! e_d^{ijx} = (v_d^i, v_d^j, x) \in \mathcal{E}_d$.

Figure 2 presents the data dependencies graph of the program given in figure 1 (left). We can notice that the order of the nodes is very different from the control flow graph, because the purpose of this graph is to put in evidence the data dependencies of the instructions of the program.

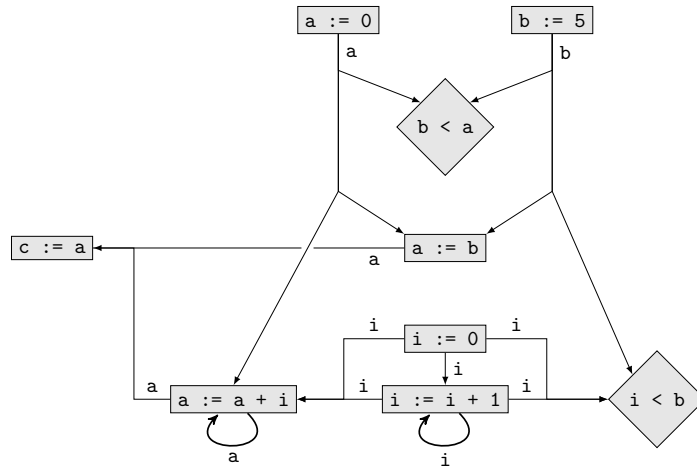


Figure 2: Data dependencies graph for the program given in figure 1 (left).

Contrary to the control flow graph, this representation already gives an intuitive idea of which instructions can be executed in parallel. For example, because instructions $a := 0$, $b := 5$ and $i := 0$ have no dependencies to be resolved, they can be executed directly at the beginning. After that, instructions $b < a$ and $a := b$ have all their dependencies resolved and can now be executed.

However, because the control flow is completely abstracted in this representation, it is not possible to distinguish an instruction that must be executed (it is executed in all the possible paths) from an instruction that may be (it is executed in only a subset of the possible paths). For example, an instruction that is under the scope of the **then** part of a test will only be executed if the test returns *true*. This may result in some ambiguous situations in which an instruction might need a single variable but from two different locations. For example, in figure 2, node $c := a$ is data-dependent on $a := b$ and $a := a + i$ about variable a .

Another limitation to this model is the fact that if we wait for an instruction to have all of its dependencies resolved before executing, then most instructions inside the body of a loop will never be able to execute because of the dependencies from themselves at a previous iteration. For example, in figure 2, instruction $i := i + 1$ is data-dependent on instructions $i := 0$ and $i := i + 1$ about variable i . Because of this auto-dependency, the instruction $i := i + 1$ will never have all of its dependency resolved, and thus will never execute.

Because of these two limitations, we need to come up with another model. To keep this idea of parallelism possibilities based on the resolution of data dependencies, this model will be based on the data dependencies model, but with additional features to be able to address the problem of ambiguous entries described previously. In the remaining of this paper, we only consider programs without loops. Addressing the more complete problem is left for future work.

1.3 Data flow graph

To introduce data flow graphs, it is first essential to present the notion of block, also called agent or actor in the literature [5, 16]. A block can be seen as a *black box* taking data as input, performing some computations, and producing data as output. It obeys the following rules:

- Any entry (or input) of an actor is identified in a unique way. For example, if an actor has two or more entries labelled with variable x , then the code represented by the actor must be able to distinguish between those inputs without any ambiguity. This can be done by renaming the entries.
- When all the inputs of an actor become available, it consumes all of them, performs some computations, and produces some results on its exits (or outputs). An actor doesn't necessary use all of its entries to perform a computation, but it still consumes them.
- When an actor has finished executing, it returns to its waiting state, until all of its entries are available again. Therefore, an actor has three distinct states: *waiting*, *ready* and *running*. However, if we make the assumption that we have infinite resources, then an actor in *ready* state will always execute immediately, and thus an actor will only have two possible states: *waiting* and *running*.

The data flow graph of a program P is a set of interconnected actors. It represents the flow of data that goes through the multiple actors when executing the program. As stated in section 1.2, we will focus in this report on loop-free programs. To compute the data flow graph of a given program, we will proceed in three steps:

1.3.1 Decomposition of the program into elementary instructions

In the remaining of this report, we assume that program instructions can be split into elementary instructions without any overhead³ at runtime. With this assumption, we can always split a compound instruction into a set of small assembly-like instructions of the form: $\text{var}_d := \text{var}_{s1} [\bullet \text{var}_{s2}]$, where $\bullet$ is any binary or arithmetic operation. This assumption is quite restrictive and will be relaxed in future work by packing instructions together after the graph transformations presented in section 2.2. With this separation into elementary instructions, any actor will have at most two entries and one exit. This transformation simplifies the analyses performed in section 2.2, because it allows us to consider a reduced number of patterns in the graph.

To do so, we proceed the same way that a boolean/arithmetic evaluator would do [17]: by decreasing nesting depth and operator priority, we assign a new variable by creating an elementary instruction of the form $\text{var}_d := \text{var}_{s1} \bullet \text{var}_{s2}$, then we inject var_d into the instruction to split instead of $\text{var}_{s1} \bullet \text{var}_{s2}$ and repeat the process until the instruction cannot be split anymore.

With this transformation, the two programs described in figure 3 are strictly equivalent, under the assumption that no overhead appears when splitting a compound instruction into elementary instructions.

<pre> x := a * (b + c) - d / e y := a - x z := x + y </pre>	<pre> x1 := b + c x2 := a * x1 x3 := d / e x := x2 - x3 y := a - x z := x + y </pre>
---	---

Figure 3: Code block including a compound instruction (left) and its equivalent sequence of elementary instructions (right) under the assumption that no overhead appears due to the decomposition.

³An overhead is an additional time resulting from something. In this particular case, the overhead could be the time for the output data of actor n to reach the input of actor n' .

1.3.2 Graph base using the data dependencies

As stated in section 1.2, the data dependencies graph already puts in evidence a lot of parallelism possibilities. For this reason, it is natural to use it as a base for the construction of the data flow graph. The translation from a data dependencies graph to a data flow graph base is very simple: every node in the data dependencies graph will result in an actor in the data flow graph, and any edge between two nodes will result in a similar edge between the two corresponding actors.

In a more formal way, let $\mathcal{G}_d = (\mathcal{V}_d, \mathcal{E}_d)$ be the data dependencies graph for a given program $P = (I, X)$. The data flow graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ of P is built as follows (elements relating to the data flow graph will have no index at all):

- $\mathcal{V}$ is the set of actors of the graph $\mathcal{G}$.
- $\forall v_d^i \in \mathcal{V}_d : \exists! v^i \in \mathcal{V}$.
- $\mathcal{E}$ is the set of edges of the graph $\mathcal{G}$. An edge $e^{ijx} \in \mathcal{E}$ from $v^i \in \mathcal{V}$ to $v^j \in \mathcal{V}$ labelled with variable $x \in X$ is a tuple (v^i, v^j, x) .
- $\forall e_d^{ijx} \in \mathcal{E}_d : \exists! e^{ijx} \in \mathcal{E}$.

Thanks to the decomposition described in section 1.3.1, $\mathcal{G}$ is based on an assembly-like version of P . This allows every actor to be the representation of a single elementary instruction, taking as input the variables it uses, and producing as output the variable it sets. Figure 4 represents the data flow graph base of the two code blocks presented in figure 3. The graph on the left is the representation of the code before the splitting step is performed, and the one on the right is the representation of the split code.

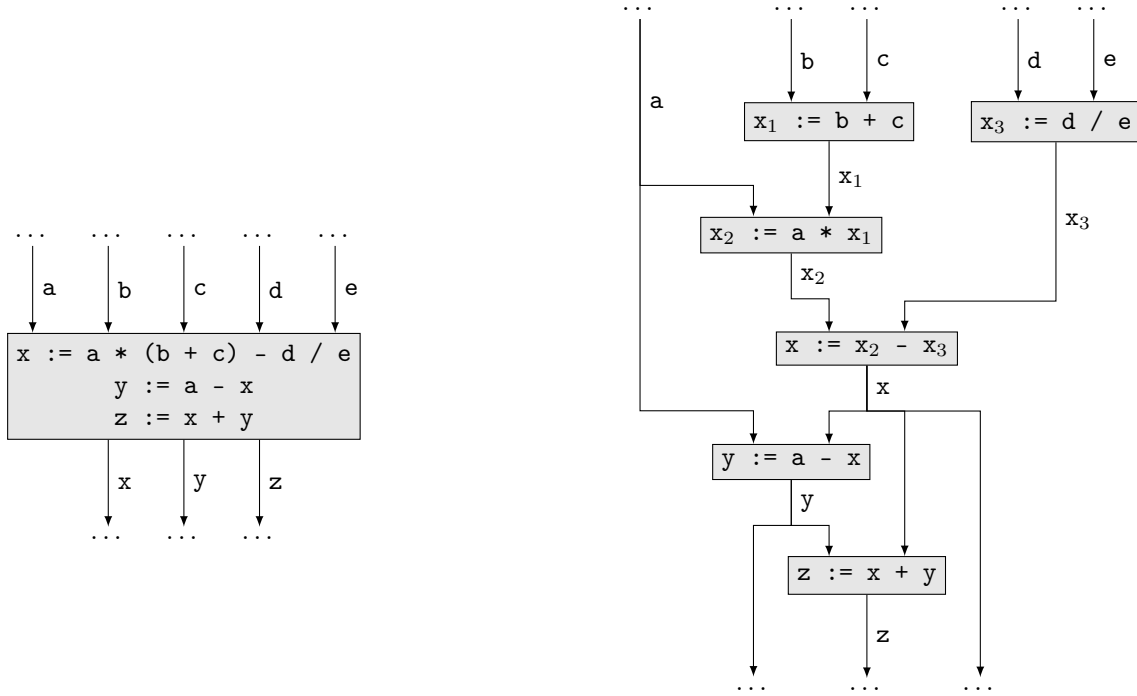


Figure 4: Data flow graphs corresponding to the two code blocks presented in figure 3.

1.3.3 The problem of ambiguous entries

As stated in the definition of an actor presented in section 1.3, all entries must be identified in a unique way. However, because the data flow graph is currently built out of a data dependency graph, this condition is not yet filled. The code presented in figure 5 illustrates this problem.

In this example, we can see that the actors representing instructions $h := a * 2$ and $i := a / h$ have two entries labelled with variable a , because they can execute with any of them depending on the context. However, by looking at the graph, there is no possibility to say which of the two possible a will be used in which situation.

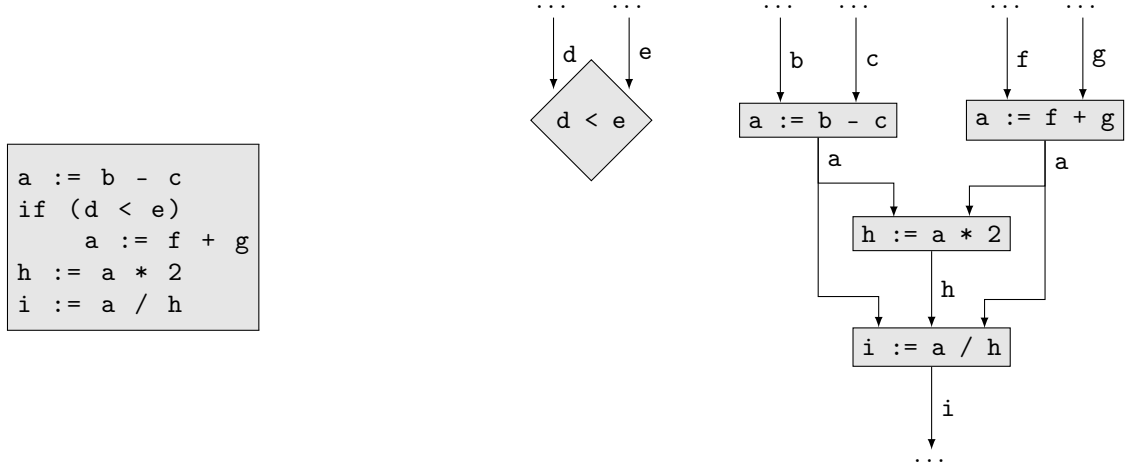


Figure 5: Code from which an ambiguity is created (left) and its data flow graph base (right).

To address this problem of ambiguous entries, we need to introduce another type of node in the data flow graph in addition to the tests and the instructions: the selectors. The idea behind a selector is, for a given set of possible inputs (variables) and a given set of conditions (boolean values resulting from tests), to forward as output one of the given inputs. This is very close to a selector in electronics, but generalized for any number of inputs and any number of conditions.

We define a selector as an actor representing a function of its data and condition entries. Therefore, a general representation of a selector is given in figure 6. In this representation, condition edges are drawn as dashed edges to distinguish them from the data edges. In addition, every input of a selector must be clearly identified to fit the data flow graph model. To achieve this, the entries are renamed internally to be unique.

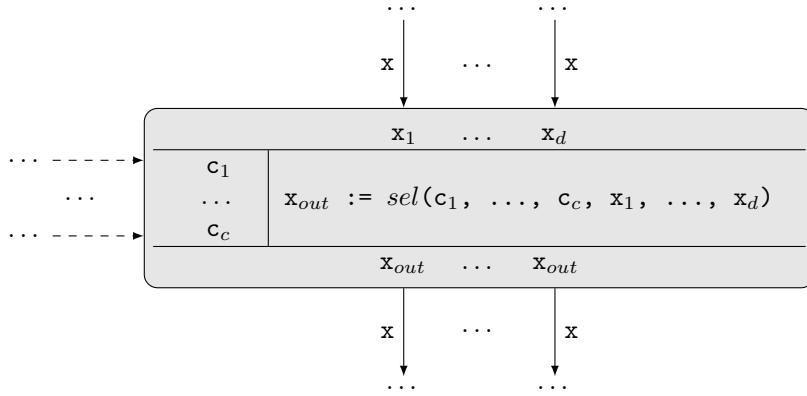


Figure 6: General representation of a selector.

In more details, a selector for a variable $x \in X$ is defined by a tuple (d, c, sel) with:

- $d \in \mathbb{N}$ the number of data inputs of the selector, $d \geq 2$.
- $c \in \mathbb{N}$ the number of condition inputs of the selector, $c \geq \log_2(d)$.
- sel a function that selects an input from $(x_m)_{1 \leq m \leq d}$ to forward to the outputs, using some of the conditions $(c_n)_{1 \leq n \leq c}$. For example, an instantiation of a selector for variable x with two data entries and a condition can be $(2, 1, \text{if } (c_1) \text{ then } x_1 \text{ else } x_2)$.

We now need to insert selectors in the data flow graph. To do so, for all node $v^i \in \mathcal{V}$ having multiple entries labelled with the same variable $x \in X$, we add a selector v^j to $\mathcal{V}$ having as inputs the inputs labelled by x of v^i , and producing as output one of the possible x that becomes the only x -labelled entry for v^i . The algorithm 1 details the insertion of the selectors in the data flow graph⁴.

⁴In our algorithms, an underlined function is a side function that won't be presented, but which name is clear enough to understand its behavior without any ambiguity. For example, the function inputEdgesOfNodeWithLabel(v^i, x) will return the edges having node v^i at their sink, labelled with variable x .

Algorithm 1: addSelectors ($\mathcal{G} = (\mathcal{V}, \mathcal{E}), P = (I, X)$)

```
foreach  $v^i \in \mathcal{V}$  do
  foreach  $x \in X$  do
     $inputs^{ix} := \text{inputEdgesOfNodeWithLabel}(v^i, x)$ 
    if  $\#inputs^{ix} > 1$  then
       $v^j := \text{new SelectorNode}()$ 
       $\mathcal{V} := \mathcal{V} \cup \{v^j\}$ 
      foreach  $e^{kix} \in inputs^{ix}$  do
         $\mathcal{E} := \mathcal{E} \setminus \{e^{kix}\}$ 
         $e^{kjx} := \text{new DataEdge}(v^k, v^j, x)$ 
         $\mathcal{E} := \mathcal{E} \cup \{e^{kjx}\}$ 
       $e^{jix} := \text{new DataEdge}(v^j, v^i, x)$ 
       $\mathcal{E} := \mathcal{E} \cup \{e^{jix}\}$ 
  return  $\mathcal{G}$ 
```

We now have to determine the tests that will provide the conditions used by the selector v^j . For every predecessor v^k of v^j in the data flow graph $\mathcal{G}$, we find all the test nodes in a possible path in the control flow graph $\mathcal{G}_c$ from the input node to v^k . We then compute the disjoint union $tests_c^j$ of all those sets. This union set now contains all test nodes from which there is at least an execution path in $\mathcal{G}_c$ to v^i .

However, if one of the test nodes $v_c^1 \in tests_c^j$ has under the scope of one of its branches (**then** or **else**) all the $(v_c^{kn})_n$ (corresponding nodes in the control flow graph of the predecessors $(v^{kn})_n$ of v^j), then it will never cause an ambiguous situation for variable x at node v^i . In this case, we remove v_c^1 from $tests_c^j$. This is justified because if v_c^1 has all $(v_c^{kn})_n$ under the scope of one of its branches, then the selector will also be under this scope, and then choosing between **true** or **false** for v_c^1 will be equivalent to choosing to possibly execute v^i or not. Additionally, if v_c^1 has no element of $(v_c^{kn})_n$ under any of its branch, we remove it too from $tests_c^j$ (the test has nothing to do with the selector). Finally, for every test node $v_c^1 \in tests_c^j$, we add a condition edge from its equivalent v^1 in the data flow graph to v^j . The addition of those condition edges is detailed in algorithm 2.

Algorithm 2: addConditionEdges ($\mathcal{G} = (\mathcal{V}, \mathcal{E}), \mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$)

```
 $selectors := \text{allSelectors}(\mathcal{V})$ 
foreach  $v^j \in selectors$  do
   $data^j := \text{allDataPredecessors}(v^j, \mathcal{G})$ 
   $data_c^j := \text{equivalent}(data^j, \mathcal{G}_c)$ 
   $tests_c^j := \{\}$ 
  foreach  $v_c^k \in data_c^j$  do
     $paths_c^k := \text{allPathsFromInputTo}(v_c^k, \mathcal{G}_c)$ 
     $tests_c^k := \text{allTestNodesInPaths}(paths_c^k)$ 
    foreach  $test_c^k \in tests_c^k$  do
       $then_c^k := \text{allNodesUnderScopeOfBranch}(test_c^k, \text{then})$ 
       $else_c^k := \text{allNodesUnderScopeOfBranch}(test_c^k, \text{else})$ 
      if  $data_c^j \not\subseteq then_c^k$  and  $data_c^j \not\subseteq else_c^k$  and  $data_c^j \cap (then_c^k \cup else_c^k) \neq \emptyset$  then
         $tests_c^j := tests_c^j \cup \{test_c^k\}$ 
  foreach  $v_c^1 \in tests_c^j$  do
     $v^1 := \text{equivalent}(v_c^1, \mathcal{G})$ 
     $e^{1j\emptyset} := \text{new ConditionEdge}(v^1, v^j, \emptyset)$ 
     $\mathcal{E} := \mathcal{E} \cup \{e^{1j\emptyset}\}$ 
  return  $\mathcal{G}$ 
```

Now, all that is left to do is to define the *sel* function of the selector v^j . This function can in fact be seen as a truth table having columns for all the test nodes connected to v^j . Then, for every row of the table, we consider that only the corresponding branch in the control flow graph $\mathcal{G}_c$ is feasible (either the **then** branch or the **else** one). With those restrictions, by following any possible path in $\mathcal{G}_c$ from an input to v^i , we keep the last predecessor v^k of v^j we find in the path, and assign it as the result of the row in the truth table. The detailed definition of the truth table is given in algorithm 3.

Algorithm 3: addSelectionFunctions ($\mathcal{G} = (\mathcal{V}, \mathcal{E})$, $\mathcal{G}_e = (\mathcal{V}_e, \mathcal{E}_e)$)

```

selectors := allSelectors( $\mathcal{V}$ )
foreach  $v^j \in \text{selectors}$  do
   $data^j := \text{allDataPredecessors}(v^j, \mathcal{G})$ 
   $conds^j := \text{allConditionPredecessors}(v^j, \mathcal{G})$ 
   $table^j := \text{new TruthTable}(conds^j)$ 
   $v^i := \text{onlySuccessor}(v^j, \mathcal{G})$ 
  foreach  $row^j \in table^j$  do
     $\mathcal{G}_{e'} := \text{copy}(\mathcal{G}_e)$ 
     $v_{e'}^i := \text{equivalent}(v^i, \mathcal{G}_{e'})$ 
    foreach  $v^1 \in conds^j$  do
       $v_{e'}^1 := \text{equivalent}(v^1, \mathcal{G}_{e'})$ 
      if  $row^j[v^1]$  then
         $\text{removeEdgeInGraph}(v_{e'}^1, \text{else}, \mathcal{G}_{e'})$ 
      else
         $\text{removeEdgeInGraph}(v_{e'}^1, \text{then}, \mathcal{G}_{e'})$ 
     $path_{e'}^i := \text{anyPathFromInputTo}(v_{e'}^i, \mathcal{G}_{e'})$ 
     $data_{e'}^j := \text{equivalent}(data^j, \mathcal{G}_{e'})$ 
     $lastAssignment_{e'}^i := \text{firstElementInSetFoundStartingFromEndOfPath}(path_{e'}^i, data_{e'}^j)$ 
    /* If no element can be found, then it means that the variable selected by  $v^j$  can be used before
    it is assigned (there is an error in P, an exception should be raised) */
     $lastAssignment^i := \text{equivalent}(lastAssignment_{e'}^i, \mathcal{G})$ 
     $table^j[row^j] := lastAssignment^i$ 
   $\text{setSelectionFunction}(v^j, table^j)$ 
return  $\mathcal{G}$ 

```

Figure 7 represents the disambiguated data flow graph for the code presented in figure 5 (left). In this example, the variable causing the ambiguity was a for instructions $h := a * 2$ and $i := a / h$, but the introduction of selectors now allows those instructions to be represented with only one a -labelled input. Moreover, the *sel* function for both selectors is the same, and the code $a_{out} := \text{if } (c_1) \text{ then } a_2 \text{ else } a_1$ is the literal translation of the truth table on the right.

c_1	a_{out}
0	a_1
1	a_2

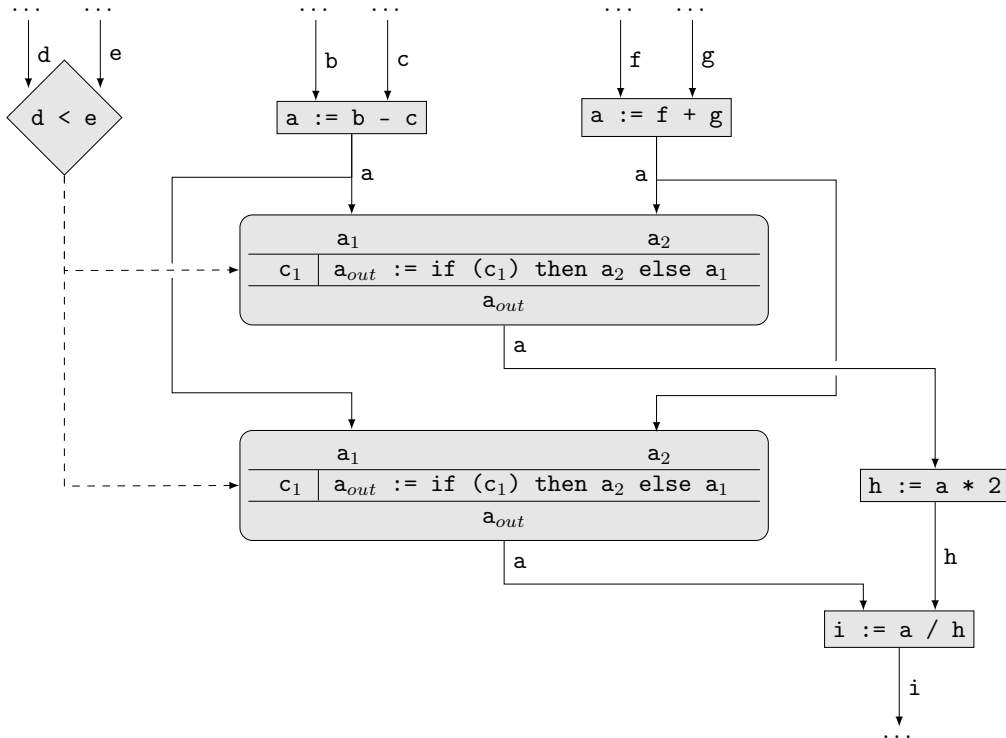


Figure 7: Disambiguated data flow graph for the program in figure 5 (left).

Finally, when all selectors are added to the data flow graph, and no ambiguity remains, we can merge the selectors that have the same inputs (data and conditions) and functions. Having two or more similar selectors in the graph only means that a variable which definition is ambiguous is used multiple times. Indeed, performing multiple times the same selection would be a loss of time. To do so, we keep only one of those similar selectors, and set its outputs as the union of all the outputs of the similar selectors. The algorithm 4 describes the merging process. To illustrate this transformation, the data flow graph in figure 8 represents the merged version of the one in figure 7.

Algorithm 4: mergeSelectors ($\mathcal{G} = (\mathcal{V}, \mathcal{E})$)

```

selectors := allSelectors( $\mathcal{V}$ )
foreach  $v^j \in selectors$  do
   $data^j := \text{allDataPredecessors}(v^j, \mathcal{G})$ 
   $conds^j := \text{allConditionPredecessors}(v^j, \mathcal{G})$ 
   $sel^j := \text{selectorFunction}(v^j)$ 
   $X_j := \text{outputVariable}(v^j)$ 
  foreach  $v^{j'} \in selectors \setminus \{v^j\}$  do
     $data^{j'} := \text{allDataPredecessors}(v^{j'}, \mathcal{G})$ 
     $conds^{j'} := \text{allConditionPredecessors}(v^{j'}, \mathcal{G})$ 
     $sel^{j'} := \text{selectorFunction}(v^{j'})$ 
    if  $data^j = data^{j'}$  and  $conds^j = conds^{j'}$  and  $sel^j = sel^{j'}$  then
       $outputs^{j'} := \text{allOutputEdges}(v^{j'}, \mathcal{G})$ 
      foreach  $e^{j'i'}(X_j) \in outputs^{j'}$  do
         $\mathcal{E} := \mathcal{E} \setminus \{e^{j'i'}(X_j)\}$ 
         $e^{ji'}(X_j) := \text{new DataEdge}(v^j, v^{i'}, X_j)$ 
         $\mathcal{E} := \mathcal{E} \cup \{e^{ji'}(X_j)\}$ 
       $\mathcal{E} := \mathcal{E} \setminus data^{j'}$ 
       $\mathcal{E} := \mathcal{E} \setminus conds^{j'}$ 
       $\mathcal{V} := \mathcal{V} \setminus \{v^{j'}\}$ 
return  $\mathcal{G}$ 

```

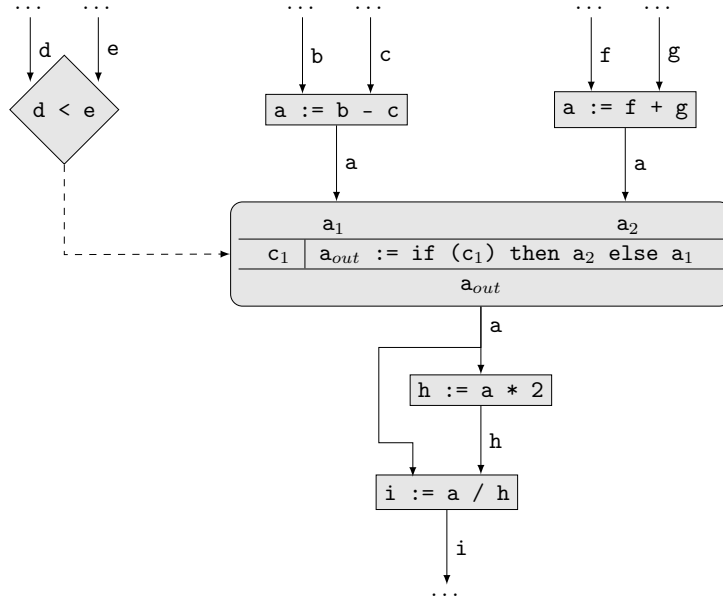


Figure 8: Merged version of the data flow graph in figure 7.

In this section, we introduced three possible representations for a given program: the control flow graph represents the possible execution paths on a sequential machine; the data dependencies graph puts in evidence which computations must be done first for the data to be ready for other computations; and the data flow graph takes profit of the parallelism indications of the data dependencies graph, while correcting the problem of ambiguous entries by the insertion of selectors. This latter model will be the base model of our improvements.

2 Transformation of the data flow graph

In this section, we present the improvements we make in the graph to reduce the WCET of a program. We first introduce in section 2.1 a technique to analyze a given node in the graph, and determine if a transformation would be profitable. Then, in section 2.2, we present our algorithm to apply changes to a spotted node and improve the execution time in the worst case. Finally, in section 2.3, we introduce the full algorithm to reduce the WCET of the whole program.

We assume here that for a given program $P = (I, X)$ we have built its control flow graph $\mathcal{G}_c$ and its data flow graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ using the techniques described in section 1.

2.1 Analysis of a given selector

To apply some transformations, we first need to determine where in the graph they should occur to be able to reduce the WCET of the program. To do so, we consider the following question: is there any location in the graph where all data entries of a selector could be available, but not all of its condition entries? This situation would undoubtedly result in some time lost waiting for the missing information to become available.

We define the worst-case termination time (WCTT) of an instruction (or of the node representing it) as the maximum time at which it finishes executing, or equivalently as the highest time at which its outputs are produced. To compute this time, we analyze the possible execution paths on a parallel machine. Therefore, the WCTT of a node $v^i \in \mathcal{G}$ in the data flow graph will be noted W^i . To simplify the analysis of the WCTT, we will assume in this work that every elementary instruction (every node) can be executed in exactly one⁵ time unit t . With this assumption, we can define W^i as the length of the longest path to v^i in $\mathcal{G}$.

In this section, we will use the program presented in figure 9 as a base for all examples. In this figure, the nodes are displayed in different rows. Each row represents a unit of time of length t . For example, node $v^1 \equiv e := a + d$ in the second row indicates that its WCTT $W^1 = 2t$. Variables b and c are available at the beginning of the program, and therefore have a null WCTT.

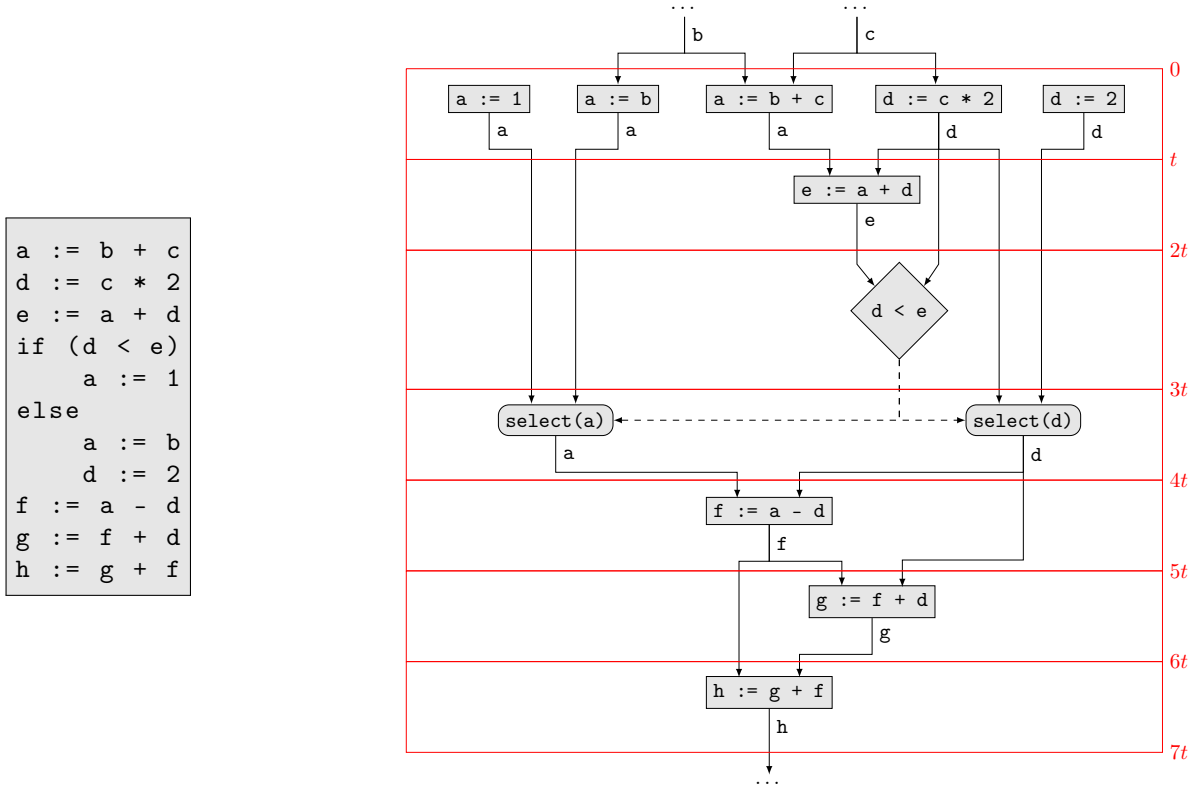


Figure 9: Example code (left) and its corresponding data flow graph (right).

The WCTT of a node $v^i \in \mathcal{G}$ is computed by the following formula, with $\text{pred}(v^i)$ defined as the set of predecessors of node v^i in the data flow graph $\mathcal{G}$.

⁵This assumption will be relaxed in later work, to introduce varying-time instructions and loops.

$$W^i = \begin{cases} t & \text{if } \#pred(v^i) = 0 \\ \max_{v^j \in pred(v^i)} (W^j) + t & \text{otherwise} \end{cases}$$

However, $pred(v^i)$ can be split into two subsets: $pred_data(v^i)$ containing the predecessors providing some data to node v^i ; and $pred_conds(v^i)$ containing the predecessors giving condition information (this set is empty except for selectors). Therefore, the previous formula becomes:

$$W^i = \begin{cases} t & \text{if } \#pred(v^i) = 0 \\ \max \left(\max_{v^j \in pred_data(v^i)} (W^j), \max_{v^k \in pred_conds(v^i)} (W^k) \right) + t & \text{otherwise} \end{cases}$$

Those two formulas are strictly equivalent, but the distinction between those two sets allows us to compute the time possibly lost at a given selector. Let v^i be a selector in the data flow graph $\mathcal{G}$. We call L^i the lost time for selector v^i . It is defined by the following formula:

$$L^i = \max_{v^k \in pred_conds(v^i)} (W^k) - \max_{v^j \in pred_data(v^i)} (W^j)$$

This value represents the difference between the arrival date of the last condition entry and the last data entry. If L^i is positive, it means that it is possible that all data entries might be available, but there would be not enough selection information to select the correct one. This would result in a loss of time of at most L^i units of time. Otherwise, if L^i is negative or null, it means that all the selection inputs will be available before or at the same time as the data inputs.

For example, in figure 9, the respective time losses of the selectors $v^i \equiv \text{select(a)}$ and $v^j \equiv \text{select(d)}$ are $L^i = 3t - \max(t, t) = 2t$ and $L^j = 3t - \max(t, t) = 2t$. This means that in the worst case, we would have to wait two units of time before we know which value is the correct one.

Using the algorithm 5, we determine for a given selector $v^i \in \mathcal{V}$ if it might have a positive lost time. This algorithm will be used by the global algorithm to minimize the WCET of the considered program.

Algorithm 5: selectorMightWait ($v^i, \mathcal{G} = (\mathcal{V}, \mathcal{E})$)

```

datai := allDataPredecessors( $v^i, \mathcal{G}$ )
condsi := allConditionPredecessors( $v^i, \mathcal{G}$ )
datumMaxWi := maxW( $data^i$ )
condMaxWi := maxW( $conds^i$ )
lossi := condMaxWi - datumMaxWi
return ( $loss^i > 0$ )

```

We now have a measure to determine for a given selector if it might suffer from a loss of time. What is left now is to address this problem by reducing this loss until it disappears. The technique to do so is presented in section 2.2.

2.2 Transformation of an improvable selector

We have shown in section 2.1 that a selector could suffer from a time loss, resulting in waiting time doing nothing because not enough information is available to continue processing. An idea to address this problem is to fill this unused time with the next possible instructions in all the possible contexts.

This technique consisting in executing in advance different branches of the code is a form of speculative execution. It is generally used in microprocessors together with branch prediction systems to execute some instructions, even if they might not be needed [18, 19]. In our case, we want to transform the graph by duplicating the sequential part after the selector in all the possible branches before the selector. The approach is then different from typical speculative execution mostly because it is performed offline, while branch predictors try to execute some code at runtime and perform a rollback in case of error.

To do so, we consider a selector $v^i \in \mathcal{V}$ for which we have detected that its L^i is strictly positive using algorithm 5. We then consider all of its successors and check for each v^j of them if its other parent⁶ v^k matches one of the following patterns:

⁶We can ensure that v^j has only two parents due to the separation in elementary actors presented in section 1.3.1.

- v^k is not a selector node. In this case, if the output of v^k is available at least one time unit before it is needed by v^i , then we can compute v^j with every possible couple of inputs $(v^{Di}, v^k) \in (v^{Di_n})_n \times \{v^k\}$ as soon as they become available, with $(v^{Di_n})_n$ being the data predecessors⁷ of v^i . We then add a selector v^1 to choose between the newly inserted v^j s (nodes $(v^{j'_m})_m$ in the graph). Figure 10 shows an example of a pattern in which this transformation can occur and the corresponding transformed graph.

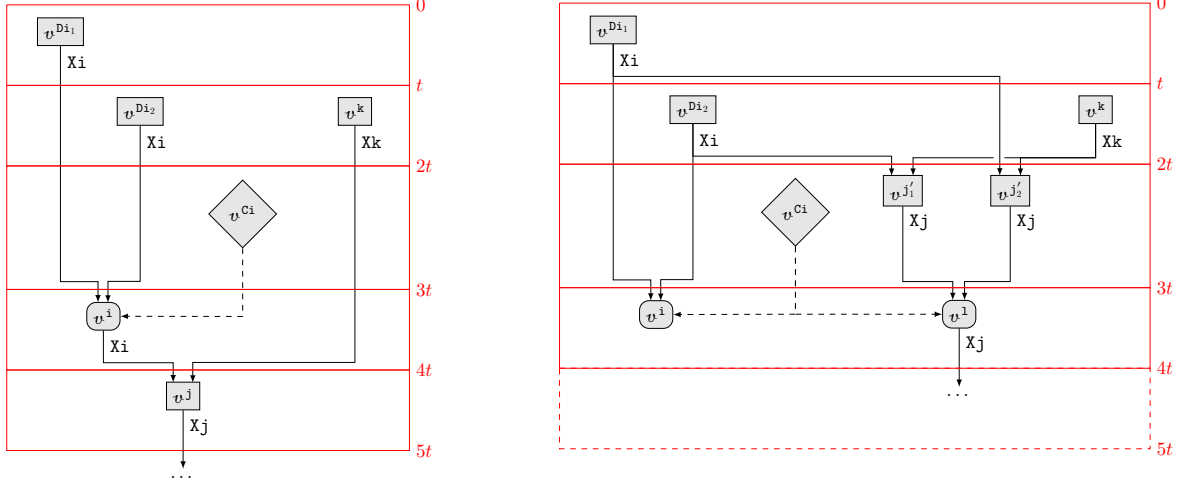


Figure 10: First transformable pattern (left) and its transformed version (right): v^k and all data predecessors of v^i are available at $2t$, so there is an unused unit of time from $2t$ to $3t$ in which v^j can be moved.

- v^k is a selector node with data predecessors $(v^{Dk_m})_m$. In this case we can apply a transformation in one of the following cases: L^k is strictly positive (there is some unused time between the element of $(v^{Di_n})_n \cup (v^{Dk_m})_m$ with maximum W , and the element of $\{v^i, v^k\}$ with maximum W); or L^k is negative or null, but v^k is available before v^i (because L^i is assumed strictly positive, then there is an unused time frame between the element of $(v^{Dk_m})_m$ with maximum W and v^i). In those cases, we can compute v^j with all possible couple of inputs $(v^{Di}, v^{Dk}) \in (v^{Di_n})_n \times (v^{Dk_m})_m$ as soon as they become available. We then add a selector v^1 to choose between the newly inserted v^j s (nodes $(v^{j'_e})_e$ in the graph). Figure 11 shows an example of a pattern in which this transformation can occur and the corresponding transformed graph.

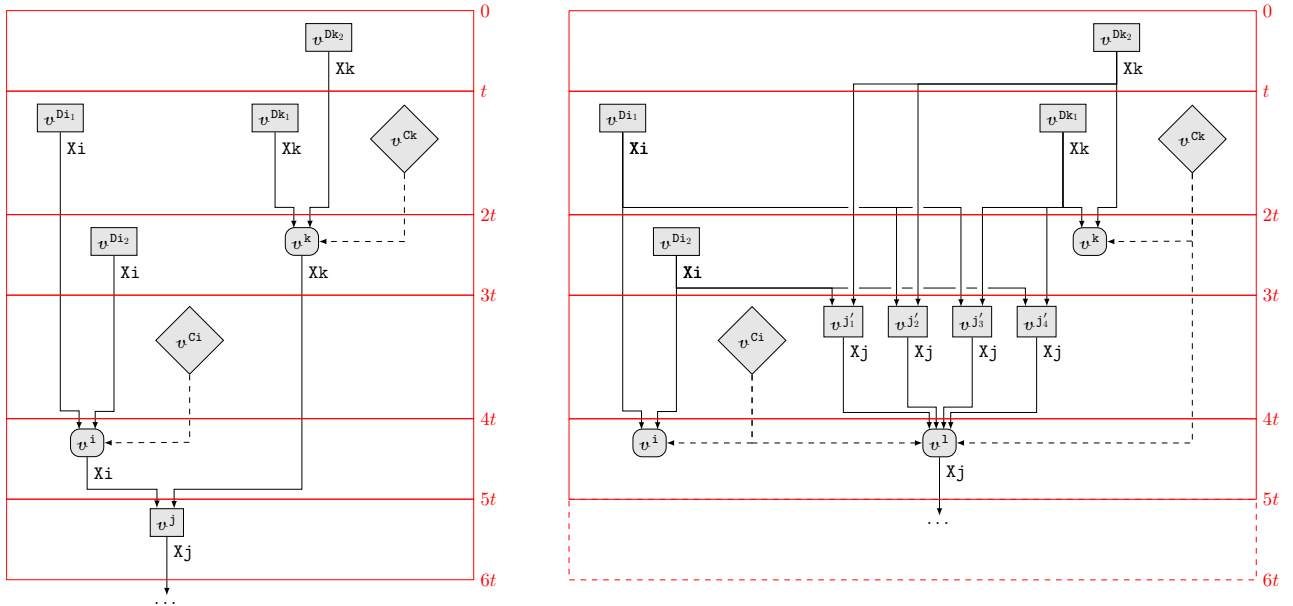


Figure 11: Second transformable pattern (left) and its transformed version (right): v^k is a selector node and all of its data predecessors are available at $2t$. Similarly, all data predecessors of v^i are available at $3t$, so there is an unused unit of time from $3t$ to $4t$ in which v^j can be moved.

⁷In the notations, D stands for "data" and C for "control". For example, v^{Di2} is the second data predecessor of v^i .

In this example, we can notice that we could also apply the first type of transformation. However, this second transformation also allows to remove some work from the selector v^k , what wouldn't be the case if we used the output of v^k to compute the new instantiations of v^j instead of using its predecessors $(v^{Dk_m})_m$ as it is the case here.

The figures 10 and 11 show that after the transformations, there might be some selectors without any output being used by other nodes. This is the case for v^i in figure 10 and for v^i and v^k in figure 11. Those selectors are no longer useful and can therefore be removed from the graph. This is illustrated by the call to algorithm 6 when the transformation at selector v^i is done by the algorithm 7.

Algorithm 6: removeSelectorIfUseless ($v^i, \mathcal{G} = (\mathcal{V}, \mathcal{E})$)

```

succsi := allSuccessors( $v^i, \mathcal{G}$ )
if #succsi = 0 then
  Xi := outputVariable( $v^i$ )
  datai := allDataPredecessors( $v^i, \mathcal{G}$ )
  foreach  $v^{Di} \in data^i$  do
     $\mathcal{E} := \mathcal{E} \setminus \{e^{(Di)i}(Xi)\}$ 
  condsi := allConditionPredecessors( $v^i, \mathcal{G}$ )
  foreach  $v^{Ci} \in conds^i$  do
     $\mathcal{E} := \mathcal{E} \setminus \{e^{(Ci)i}()\}$ 
   $\mathcal{V} := \mathcal{V} \setminus \{v^i\}$ 
return  $\mathcal{G}$ 

```

The algorithm 7 is a study of all improvable cases for a selector $v^i \in \mathcal{V}$ with $L^i > 0$. The algorithm 8 shows how to perform the improvement. The transformation basically consists in duplicating the successors $(v^{j'})_o$ of v^i for every possible couple of nodes in the data predecessors of v^i and v^k (or v^k itself if it is not a selector). When this duplication is done from a given v^j , we link the newly inserted $v^{j'}$ s to the correct data nodes, and add a selector v^1 for all the $v^{j'}$ s inserted to determine which one was the correct one.

With those transformations, we save some time in the sense that all the instructions that would be executed at time W^j (before the transformation) are now executed at time W^1 , with $W^1 < W^j$. The proof is quite simple: we know that $L^i > 0$, meaning that there is at least one time unit between the time at which all the data entries of v^i are available, and the time at which all of its entries (including condition entries) are available. The studied cases are the ones in which v^k (or its data inputs if v^k is a selector) are all available before or at the same time as the data input of v^i with maximum W . Therefore all possible executions of v^j can be done at $W^1 - 2t$ at the latest. We can then select (with the new selector v^1) in the next unit of time the correct newly inserted $v^{j'}$, meaning that its output will be available at $W^1 = W^i$, which is strictly inferior to the original value of W^j .

Algorithm 7: improveWaitingSelector ($v^i, criticalPath, \mathcal{G} = (\mathcal{V}, \mathcal{E}), \mathcal{G}_e = (\mathcal{V}_e, \mathcal{E}_e)$)

```

succsi := allSuccessorsSortedByIncreasingW( $v^i, \mathcal{G}$ )
foreach  $v^j \in succs^i$  do
  if  $v^j \in criticalPath$  then
    /* We don't consider all the possible successors in order to minimize the number of duplicated
    nodes when executing algorithm 10 */
     $v^k := otherPredecessor(v^j, v^i, \mathcal{G})$ 
    if  $v^k$  is not SelectorNode and  $W^i - W^k > 1$  then
       $\mathcal{G} := transformSelector(v^i, v^j, v^k, \{v^k\}, \mathcal{G}, \mathcal{G}_e)$  // First type of transformation (figure 10)
    else if  $v^k$  is SelectorNode and ( $L^k > 0$  or ( $L^k \leq 0$  and  $W^k < W^i$ )) then
      datak := allDataPredecessors( $v^k, \mathcal{G}$ )
       $\mathcal{G} := transformSelector(v^i, v^j, v^k, data^k, \mathcal{G}, \mathcal{G}_e)$  // Second type of transformation (figure 11)
       $\mathcal{G} := removeSelectorIfUseless(v^k, \mathcal{G})$ 
     $\mathcal{G} := removeSelectorIfUseless(v^i, \mathcal{G})$ 
return  $\mathcal{G}$ 

```

Algorithm 8: transformSelector ($v^i, v^j, v^k, otherBranch^k, \mathcal{G} = (\mathcal{V}, \mathcal{E}), \mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$)

```

Xi := outputVariable( $v^i$ )
Xj := outputVariable( $v^j$ )
Xk := outputVariable( $v^k$ )
 $\mathcal{E} := \mathcal{E} \setminus \{e^{ij}(Xi)\}$ 
 $\mathcal{E} := \mathcal{E} \setminus \{e^{kj}(Xk)\}$ 
succsj := allSuccessors( $v^j, \mathcal{G}$ )
 $v^1 := \text{new SelectorNode}(Xj)$ 
 $\mathcal{V} := \mathcal{V} \cup \{v^1\}$ 
foreach  $v^m \in \text{succs}^j$  do
   $\mathcal{E} := \mathcal{E} \setminus \{e^{jm}(Xj)\}$ 
   $e^{1m}(Xj) := \text{new ConditionEdge}(v^1, v^m, Xj)$ 
   $\mathcal{E} := \mathcal{E} \cup \{e^{1m}(Xj)\}$ 
 $\mathcal{V} := \mathcal{V} \setminus \{v^j\}$ 
datai := allDataPredecessors( $v^i, \mathcal{G}$ )
foreach ( $v^{Di}, v^{Dk}$ )  $\in \text{couples}(v^j, data^i, Xi, otherBranch^k, Xk, \mathcal{G}, \mathcal{G}_c)$  do
  /* For the first read, consider that function couples returns the Cartesian product of datai and
  otherBranchk. It will be redefined later to solve the problem of nodes that shouldn't be inserted */
   $v^{j'} := \text{new InstructionNode}(v^j)$ 
   $\mathcal{V} := \mathcal{V} \cup \{v^{j'}\}$ 
   $e^{(Di)j'}(Xi) := \text{new DataEdge}(v^{Di}, v^{j'}, Xi)$ 
   $\mathcal{E} := \mathcal{E} \cup \{e^{(Di)j'}(Xi)\}$ 
   $e^{(Dk)j'}(Xk) := \text{new DataEdge}(v^{Dk}, v^{j'}, Xk)$ 
   $\mathcal{E} := \mathcal{E} \cup \{e^{(Dk)j'}(Xk)\}$ 
   $e^{j'1}(Xj) := \text{new DataEdge}(v^{j'}, v^1, Xj)$ 
   $\mathcal{E} := \mathcal{E} \cup \{e^{j'1}(Xj)\}$ 
condsi := allConditionPredecessors( $v^i, \mathcal{G}$ )
foreach  $v^{Ci} \in \text{conds}^i$  do
   $e^{(Ci)1\emptyset} := \text{new ConditionEdge}(v^{Ci}, v^1, \emptyset)$ 
   $\mathcal{E} := \mathcal{E} \cup \{e^{(Ci)1\emptyset}\}$ 
condsk := allConditionPredecessors( $v^k, \mathcal{G}$ )
foreach  $v^{Ck} \in \text{conds}^k$  do
   $e^{(Ck)1\emptyset} := \text{new ConditionEdge}(v^{Ck}, v^1, \emptyset)$ 
   $\mathcal{E} := \mathcal{E} \cup \{e^{(Ck)1\emptyset}\}$ 
 $\mathcal{G} := \text{addSelectionFunction}(v^1, \mathcal{G}, \mathcal{G}_c)$  // Like function addSelectionFunction but for a given selector only
return  $\mathcal{G}$ 

```

We now have a method to improve the execution time of some code using the time loss detected at a given selector in the data flow graph, using the previous algorithm 7. However, this transformation can introduce false nodes if we use a simple Cartesian product as the definition of function *couples*. If we apply algorithm 7 on the selector $v^i \equiv \text{select}(\mathbf{a})$ from figure 9 (right), we get the data flow graph presented in figure 12.

On this figure, we can notice that the transformation at $v^i \equiv \text{select}(\mathbf{a})$ introduced four instructions $\mathbf{f} := \mathbf{a} - \mathbf{d}$ to replace the original one. Those instructions correspond to all the possible couples of variables $\mathbf{a}$ and $\mathbf{d}$ given as input of the moved instruction $\mathbf{f} := \mathbf{a} - \mathbf{d}$, according to the pattern described in figure 11. However, when reading the code in figure 9 (left) from which the data flow graph is built, we can notice that it is never possible to execute $\mathbf{f} := \mathbf{a} - \mathbf{d}$ with $\mathbf{a}$ being set by instruction $\mathbf{a} := 1$ and $\mathbf{d}$ being set by instruction $\mathbf{d} := 2$ because they are in mutually exclusive branches of the same conditional structure. Therefore, the inserted node $\mathbf{f} := \mathbf{a} - \mathbf{d}$ that is the most on the left on figure 12 shouldn't have been added to the graph. The reasoning is the same for the one that is the most on the right, because the value of $\mathbf{d}$ is inevitably set by instruction $\mathbf{d} := 2$ if $\mathbf{a}$ is set by instruction $\mathbf{a} := \mathbf{b}$.

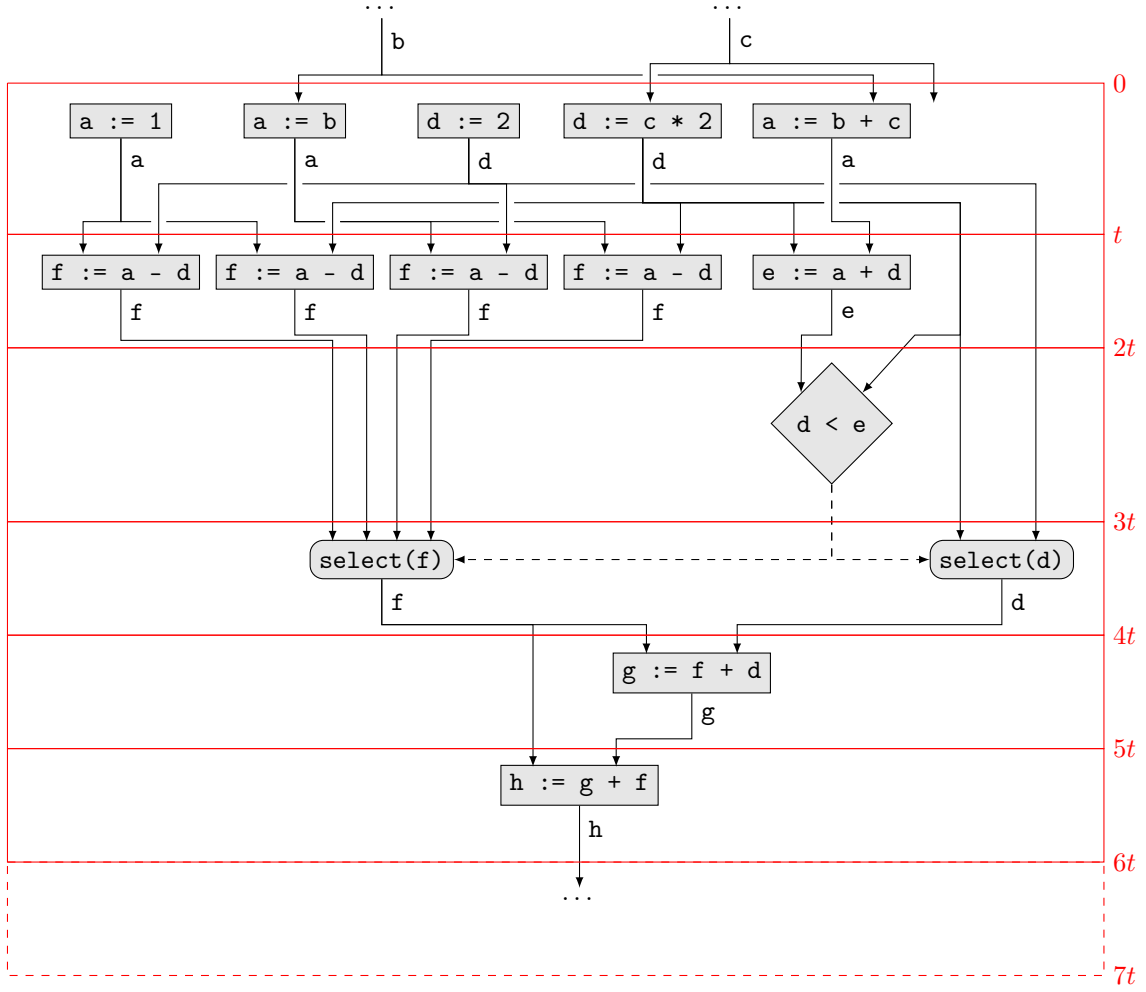


Figure 12: Data flow graph from figure 9 (right) after the application of algorithm 7 on selector for variable **a**.

To avoid this problem of inserted nodes that shouldn't exist, we will use the definition of function *couples* presented in algorithm 9 instead of a simple Cartesian product. This algorithm checks for all possible couple $(v_c^{Di}, v_c^{Dk}) \in data^i \times otherBranch^k$ if there is a path in the control flow graph $\mathcal{G}_c$ along which the considered successor v_c^j (in our example, **f := a - d**) of v_c^i can execute with inputs that can effectively be set by v_c^{Di} and v_c^{Dk} .

Algorithm 9: couples $(v^j, data^i, Xi, otherBranch^k, Xk, \mathcal{G} = (\mathcal{V}, \mathcal{E}), \mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c))$

```

possibleCouples := {}
 $v_c^j := \text{equivalent}(v^j, \mathcal{G}_c)$ 
foreach  $(v_c^{Di}, v_c^{Dk}) \in data^i \times otherBranch^k$  do
     $v_c^{Di} := \text{equivalent}(v_c^{Di}, \mathcal{G}_c)$ 
     $v_c^{Dk} := \text{equivalent}(v_c^{Dk}, \mathcal{G}_c)$ 
     $paths_c^j := \text{allPathsFromInputTo}(v_c^j, \mathcal{G}_c)$ 
    foreach  $path_c^j \in paths_c^j$  do
        if  $\text{lastAssignmentIsByNode}(path_c^j, Xi, v_c^{Di})$  and  $\text{lastAssignmentIsByNode}(path_c^j, Xk, v_c^{Dk})$  then
             $possibleCouples := possibleCouples \cup \{(v_c^{Di}, v_c^{Dk})\}$ 
            break
return possibleCouples

```

With this new version of function *couples*, we now have an algorithm 7 that improves the execution time of the data flow graph considering a selector v^i having a strictly positive L^i , while keeping the semantics of the original program. We now need to provide an algorithm that applies this technique on multiple selectors to reduce the critical path as much as possible, and thus to use all the wasted time along the worst-case execution path, reducing the worst-case execution time of the full program.

2.3 Algorithm to improve the worst-case execution time of the program

As said before, the worst-case execution time of a program is the maximum time it takes to execute. Due to our assumption in section 2.1, the execution time of one node in a graph representing the program is of a fixed length t . This means that a worst-case execution path of the analyzed program is a path from its input node to one of its output nodes including the highest number of nodes. This path is also called the critical path of the program. However, it is not unique, and since we focus on improving the WCET of the program, we have to consider all of them. In this section, we will focus on the creation of an algorithm to minimize the critical paths in the data flow graph $\mathcal{G}$ representing a program P .

A critical path can be found very easily since we work on loop-free programs thanks to the assumption presented in section 1.2. To do so, we can use an existing low-complexity ($O(|\mathcal{V}| + |\mathcal{E}|)$) algorithm presented in [20], returning one of the longest paths in a directed acyclic graph (DAG).

As we have seen previously, we can execute the same amount of instructions in less time if we can detect a selector v^i having a strictly positive L^i . The transformation we can apply is a duplication of the successors of v^i to execute them in all possible configuration of their input variables, and to choose between the different outputs to keep the correct one.

We can notice that if v^i belongs to a critical path of the graph, then at least one of its successors does too. We can then conclude that if this successor can be moved, then this longest path is reduced. With this remark, we can then focus only on the selectors that belong to a critical path of the program. Notice that we could also try to transform the totality of the selectors in the graph, but it would cause a possible explosion of the number of inserted nodes.

Another important point to consider is the fact that if we improve a selector v^i in a critical path p_1 , then the length of p_1 is reduced; and if there is another path p_2 in the graph that has the same length as p_1 before its transformation, then p_2 becomes the new longest path in the graph and has to be considered in the remaining of the algorithm.

The algorithm 10 minimizes the worst-case execution time of the data flow graph given as parameter using the techniques presented before. It processes iteratively by considering only one improvable selector on the actual critical path, and computes a new critical path to improve it, until no improvement becomes possible. It then means that the considered longest path cannot be reduced anymore using this technique; and whatever additional selector we improve, it won't reduce the WCET of the program.

Algorithm 10: minimizeWCET ($\mathcal{G} = (\mathcal{V}, \mathcal{E})$, $\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$)

```

continueLoop := true
while continueLoop do
  continueLoop := false
  criticalPath := findOneCriticalPath( $\mathcal{G}$ )
  selectors := allSelectors(criticalPath)
  foreach  $v^i \in \text{selectors}$  do
    if selectorMightWait( $v^i$ ,  $\mathcal{G}$ ) then
       $\mathcal{G}' := \text{improveWaitingSelector}(v^i, \text{criticalPath}, \mathcal{G}, \mathcal{G}_c)$ 
      if  $\mathcal{G}' \neq \mathcal{G}$  then
         $\mathcal{G} := \mathcal{G}'$ 
        continueLoop := true
        break
  return  $\mathcal{G}$ 

```

With this algorithm, the example graph presented in figure 9 becomes the data flow graph in figure 13. The algorithm first considers a possible worst-case execution path $a := b + c$; $e := a + d$; $d < e$; **select**(a); $f := a - d$; $g := f + d$; $h := g + f$, and improves at selector $v^i \equiv \text{select}(a)$ having a positive L^i . After this transformation, we have the data flow graph presented in figure 12 (without the two impossible instructions $f := a - d$, as explained in the end of section 2.2).

Then the new considered critical path becomes $a := b + c$; $e := a + d$; $d < e$; **select**(f); $g := f + d$; $h := g + f$. In this path, we consider $v^j \equiv \text{select}(f)$ and try to duplicate its successors $v^k \equiv g := f + d$ and $v^l \equiv h := g + f$. According to the transformable templates studied in section 2.2, v^k can be moved to start at $2t$, but then v^l cannot move.

Finally, the longest path studied becomes $a := b + c$; $e := a + d$; $d < e$; **select**(f); $h := g + f$. We select again $v^j \equiv \text{select}(f)$ and try to move $v^l \equiv h := g + f$. However we saw previously that it wasn't possible

because it doesn't match the improvable templates (the parent nodes of `select(g)` are computed too late to be able to save some time). There is no other selector in the critical path, so we have found one of the longest paths that cannot be reduced. Therefore the algorithm is terminated and the data flow graph cannot be improved anymore with this technique.

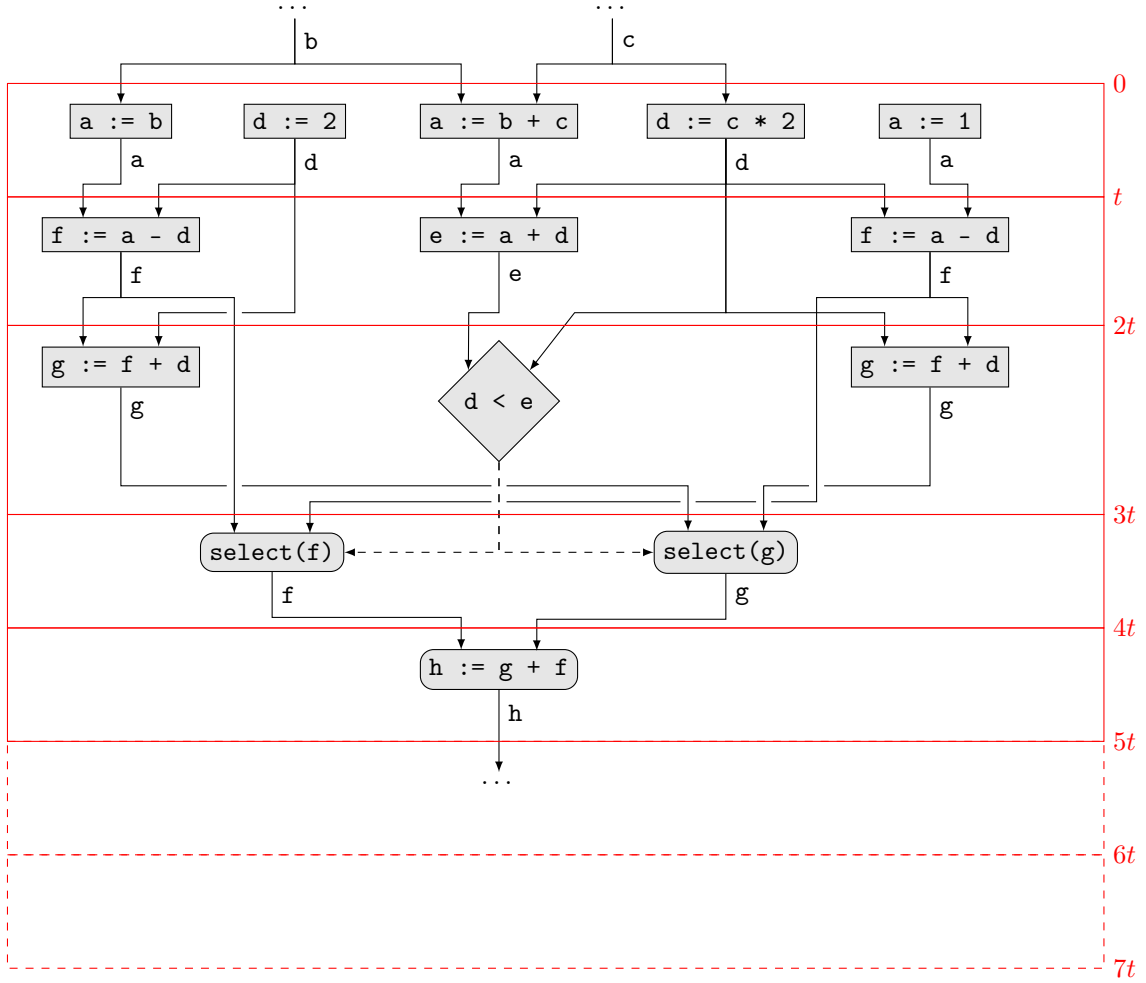


Figure 13: Data flow graph from figure 9 (right) after the application of algorithm 10.

In this section, we presented a way to detect areas in the data flow graph of a program where some time might be lost waiting for control information even if all the data to compute the remaining of the program is already available. We then explained how to take profit of this detection to reduce the execution time of a portion of code. Finally, we came up with an algorithm to minimize the size of the critical paths of a given program, and as a consequence to reduce its worst-case execution time.

Conclusion

In this report, we presented various representations for a given program: the well-known control flow graph, allowing to analyze the behavior of the program on a typical Von Neumann architecture; and the data flow graph, built from the data dependencies graph, allowing to present the program as a graph of actors and putting in evidence a lot of parallelism. Basing ourselves on the data flow graph of the program, we were able to detect some locations in the graph where all data is available, but where we must wait for control information to continue executing the remaining of the program. We then introduced a method based on code duplication and speculative execution to fill this unused time with additional computations, reducing the execution time of the code. Finally, we designed an algorithm that reduces iteratively the length of the critical path of a given program, diminishing its WCET at the same time.

This research is the first step of some further work that will be continued here. The current technique focuses on loop-free programs running on an infinite number of cores, with the assumption that every instructions takes the same time to execute. Future work will first consist in expanding the data flow model to take loops in consideration to be able to consider more programs. Then we will relax our assumptions concerning the number of cores and the execution time of the instructions to allow an implementation on the Kalray MPPA-256 many-core processor in our laboratory.

As a conclusion, we presented a technique to reduce the worst-case execution time of a given program, under some assumptions that we will relax in later work. This graph transformation method could be inserted in a compiler specialized in real-time systems, and has a lot of potential applications, such as allowing the respect of the deadline of a program on a processor with a lower frequency to avoid the need to buy a more expensive faster platform.

References

- [1] Intel Corporation. *Moore's Law Timeline*. Poster. 2005.
- [2] A. Chien. *Moore's Law: The First Ending and A New Beginning*. 2012.
- [3] R. Wilhelm; J. Engblom; A. Ermedahl et al. "The worst-case execution-time problem—overview of methods and survey of tools". In: *ACM Transactions on Embedded Computing Systems (TECS)* (2008).
- [4] S. Edgar; A. Burns. "Statistical analysis of WCET for scheduling". In: *Real-Time Systems Symposium. Proceedings. 22nd IEEE*. 2001.
- [5] Kalray Corporation. *Kalray MPPA - A New Era of Processing*. Presentation slides for the MPPA suite. 2012.
- [6] C. Moore. *Data processing in exascale-class computer systems*. Salishan Conference on High Speed Computing. 2011.
- [7] B. Barney. *POSIX Threads Programming*. <https://computing.llnl.gov/tutorials/pthreads/>. 2013, consulted on 2013/07/23.
- [8] Mathematics and Argonne National Laboratory Computer Science. *The Message Passing Interface (MPI) standard*. <http://www.mcs.anl.gov/research/projects/mpi/>. 2012, consulted on 2013/07/23.
- [9] *The OpenMP API specification for parallel programming*. <http://openmp.org/wp/>. 2013, consulted on 2013/07/23.
- [10] Indian Institute of Science. *PLUTO - An automatic parallelizer and locality optimizer for multicores*. <http://pluto-compiler.sourceforge.net/>. 2013, consulted on 2013/07/23.
- [11] F. Irigoin. "Interprocedural Analyses for Programming Environments". In: *Workshop on Environment and Tools For Parallel Scientific Computing, CNRS-SNF*. 1992.
- [12] U. Bondhugula; M. Baskaran; S. Krishnamoorthy et al. "Automatic Transformations for Communication-Minimized Parallelization and Locality Optimization in the Polyhedral Model". In: *International Conference on Compiler Construction (ETAPS CC)*. 2008.
- [13] MINES ParisTech; CEA-DAM; Southampton University et al. *Overview of PIPS*. http://pips4u.org/doc/pips_summary.html. 2013, consulted on 2013/07/23.
- [14] R. Eigenmann; D.J. Lilja. *Von Neumann Computers*. 1998.
- [15] R. Gold. "Control Flow Graphs and Code Coverage". In: *International Journal of Applied Mathematics and Computer Science* (2010).
- [16] J.L. Pino; E.A. Lee. "Hierarchical static scheduling of dataflow graphs onto multiple processors". In: *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing*. 1995.
- [17] H. Abelson; G.J. Sussman; J. Sussman. *Structure and Interpretation of Computer Programs*. MIT Electrical Engineering and Computer Science, 1996.
- [18] A. Sez nec. "Encyclopedia of Parallel Computing". In: Springer, 1996. Chap. Branch Predictors.
- [19] K. Asanovic. *Branch Prediction and Speculative Execution*. CSAIL slides. 2005.
- [20] R. Sedgewick; K. Wayne. *Algorithms*. Addison-Wesley Professional, 2011.